

INFORME DE TAREA 4

Pruebas de Rendimiento con Apache JMeter

Asignatura: INF780 – Verificación y Validación de Software

Docente: M. Sc. Huáscar Fedor Gonzales Guzmán

Estudiante: Clyder Remmy Moreira Quispe

Modalidad: Individual

Fecha: 30 de junio de 2026

1. Introducción y Objetivos

El presente informe documenta las pruebas de rendimiento (no funcionales) realizadas sobre la API REST *movies-api* desarrollada en NestJS + TypeORM + PostgreSQL. El objetivo es evaluar el comportamiento del sistema bajo distintos niveles de carga mediante Apache JMeter 5.6.3, identificar el punto de saturación y proponer mejoras.

Objetivos específicos:

- Verificar el correcto funcionamiento de la API con pocos usuarios (smoke test).
- Evaluar el comportamiento bajo carga esperada (50 usuarios concurrentes).
- Identificar el punto de quiebre mediante prueba de estrés (100/200/400 usuarios).
- Medir la reacción ante picos bruscos de tráfico (200 usuarios en 5 segundos).
- Documentar métricas: throughput, tiempos de respuesta (media, percentiles) y tasa de error.

2. Entorno de Pruebas

Componente	Versión / Detalle
Sistema Operativo	Windows 10/11 x64
Java	OpenJDK 25.0.3 (2026-04-21 LTS)
Apache JMeter	5.6.3
Node.js	v24.14.0
NestJS	10.x (movies-api)
PostgreSQL	Local (puerto 5432)
API Base URL	http://localhost:3000

Nota: Las pruebas se ejecutaron en modo no-GUI para evitar interferencia de la interfaz gráfica en las mediciones.

3. Diseño de las Pruebas

Escenario	Tipo	Usuarios	Ramp-up	Loops	Endpoints	Aserciones
Smoke	Baseline	1	1 s	5	GET /movies	—
Carga	Load	50	30 s	10	GET /movies GET /movies/{id} POST /movies	—
Estrés 100	Stress	100	30 s	10	Ídem carga	—
Estrés 200	Stress	200	30 s	10	Ídem carga	—
Estrés 400	Stress	400	30 s	10	Ídem carga	—
Picos	Spike	200	5 s	5	Ídem carga	HTTP 200/201 Duración 800ms

Se utilizó CSV Data Set Config con 64 IDs reales para parametrizar GET /movies/{id}. Un Constant Timer de 300ms simula tiempos de espera realistas entre peticiones.

4. Resultados por Escenario

4.1 Smoke Test

1 usuario virtual, 5 iteraciones. Objetivo: verificar conectividad y obtener línea base.

Endpoint	#Muestras	Media (ms)	Mín (ms)	Máx (ms)	Error %	Throughput
GET /movies	5	67	7	301	0.00%	4.9/s

TOTAL	5	67	7	301	0.00%	4.9/s
-------	---	----	---	-----	-------	-------

✓ 0% de errores. La API responde correctamente en condiciones mínimas. Línea base: 67ms promedio.

4.2 Prueba de Carga (50 usuarios)

50 usuarios, ramp-up 30s, 10 loops = 1500 peticiones totales.

Endpoint	#Muestras	Media (ms)	P90 (ms)	P95 (ms)	P99 (ms)	Error %	T/s
GET /movies	500	307	632	715	874	0.00%	10.89
GET /movies/{id}	500	191	477	547	750	0.00%	10.98
POST /movies	500	492	851	971	1150	0.00%	10.98
TOTAL	1500	330	716	822	1063	0.00%	31.2

✓ 0% errores. Throughput total 31.2/s. El endpoint POST es el más lento (492ms promedio).

4.3 Prueba de Estrés (100 / 200 / 400 usuarios)

Nivel	Peticiones	Duración	Throughput	Media (ms)	Máx (ms)	Error %
100 usuarios	3000	2:02 min	24.7/s	2568	7634	0.00%
200 usuarios	6000	8:13 min	12.2/s	13102	31114	0.78%
400 usuarios	12000	19:46 min	10.1/s	38076	67742	2.62%

Con 100 usuarios no hay errores pero el tiempo promedio sube a 2.5s. Con 200 usuarios aparecen los primeros errores (0.78%). Con 400 usuarios el sistema se degrada significativamente (2.62% errores, 38s promedio).

4.4 Prueba de Picos (200 usuarios, ramp-up 5s)

Arranque brusco: 200 usuarios en 5 segundos, 5 loops = 3000 peticiones.

Métrica	Valor
Total peticiones	3000
Duración	7:32 min
Throughput	6.6/s
Media (ms)	28594
Máximo (ms)	105969
Error % (Duration Assertion 800ms)	99.33%

El alto porcentaje de error (99.33%) es causado por la Duration Assertion de 800ms: bajo un pico brusco de 200 usuarios, prácticamente ninguna petición responde en menos de 800ms. Las aserciones de código HTTP (200/201) no generaron errores adicionales.

5. Análisis Comparativo

Escenario	Usuarios	Throughput	Media (ms)	Error %	Estado
Smoke	1	4.9/s	67	0.00%	✓ OK
Carga	50	31.2/s	330	0.00%	✓ OK
Estrés 100	100	24.7/s	2568	0.00%	■ Lento
Estrés 200	200	12.2/s	13102	0.78%	■ Degradado
Estrés 400	400	10.1/s	38076	2.62%	✗ Saturado
Picos	200	6.6/s	28594	99.33%*	✗ Colapso

* Error% en picos incluye violaciones de Duration Assertion (800ms).

Punto de saturación: Entre 100 y 200 usuarios concurrentes.

Con 100 usuarios el throughput cae de 31.2/s a 24.7/s y el tiempo promedio sube de 330ms a 2568ms (7.8x más lento), sin errores HTTP pero con degradación severa de rendimiento. A 200 usuarios aparecen los primeros errores y el promedio supera los 13 segundos, indicando que la API ha alcanzado su límite operativo aceptable.

Cuellos de botella identificados:

- **Base de datos:** Sin índices en columnas de búsqueda frecuente (genre, director, year). Las consultas se vuelven full table scans bajo carga.
- **Pool de conexiones:** TypeORM usa un pool limitado; con 200+ usuarios concurrentes las peticiones esperan conexión disponible.
- **Endpoint POST /movies:** Es consistentemente el más lento (492ms en carga normal) por la operación de escritura + validación.
- **Sin caché:** Cada GET /movies consulta la BD completa sin reutilizar resultados anteriores.

6. Conclusiones y Recomendaciones

Conclusiones:

1. La API funciona correctamente bajo carga moderada (hasta 50 usuarios), con 0% de errores y tiempos aceptables.
2. El punto de saturación se encuentra entre 100 y 200 usuarios: los tiempos de respuesta se degradan drásticamente y aparecen errores.
3. Ante picos bruscos de tráfico, la API no logra recuperarse dentro del umbral de 800ms, lo que indica falta de mecanismos de resiliencia.
4. El throughput máximo sostenible observado es ~31 req/s con 50 usuarios concurrentes.

Recomendaciones técnicas:

1. **Índices en PostgreSQL:** Crear índices en columnas genre, director y year para acelerar las consultas frecuentes:
`CREATE INDEX idx_movies_genre ON movies(genre);`
2. **Caché HTTP con Redis:** Implementar caché para GET /movies con TTL de 30-60 segundos. Esto reduciría la carga en BD en un 60-80% bajo carga alta.
3. **Paginación obligatoria:** El endpoint GET /movies devuelve todos los registros. Implementar paginación (limit/offset o cursor) para limitar el tamaño de respuesta.
4. **Pool de conexiones:** Aumentar el pool de TypeORM a 20-30 conexiones para soportar mayor concurrencia.
5. **Rate limiting:** Implementar throttling con @nestjs/throttler para proteger la API de picos de tráfico no controlados.

7. Anexos

7.1 Archivos .jmx entregados

Archivo	Descripción	Ubicación
smoke.jmx	Smoke test - 1 usuario	jmeter/
carga.jmx	Prueba de carga - 50 usuarios	jmeter/
estres.jmx	Prueba de estrés - 100/200/400	jmeter/
picos.jmx	Prueba de picos - 200u ramp 5s	jmeter/

7.2 Archivos de resultados .jtl

Archivo	Escenario	Peticiones	Ubicación
smoke.jtl	Smoke test	5	resultados/
carga.jtl	Carga 50u	1500	resultados/
estres-100.jtl	Estrés 100u	3000	resultados/
estres-200.jtl	Estrés 200u	6000	resultados/
estres-400.jtl	Estrés 400u	12000	resultados/
picos.jtl	Picos 200u	3000	resultados/

7.3 Reportes HTML generados

Cada corrida generó un dashboard HTML completo con gráficos de throughput, tiempos de respuesta y percentiles, ubicados en las carpetas: smoke-report/, carga-report/, estres-report-100/, estres-report-200/, estres-report-400/, picos-report/ dentro del directorio jmeter/.